Защищённая оверлейная сеть на основе Kademlia

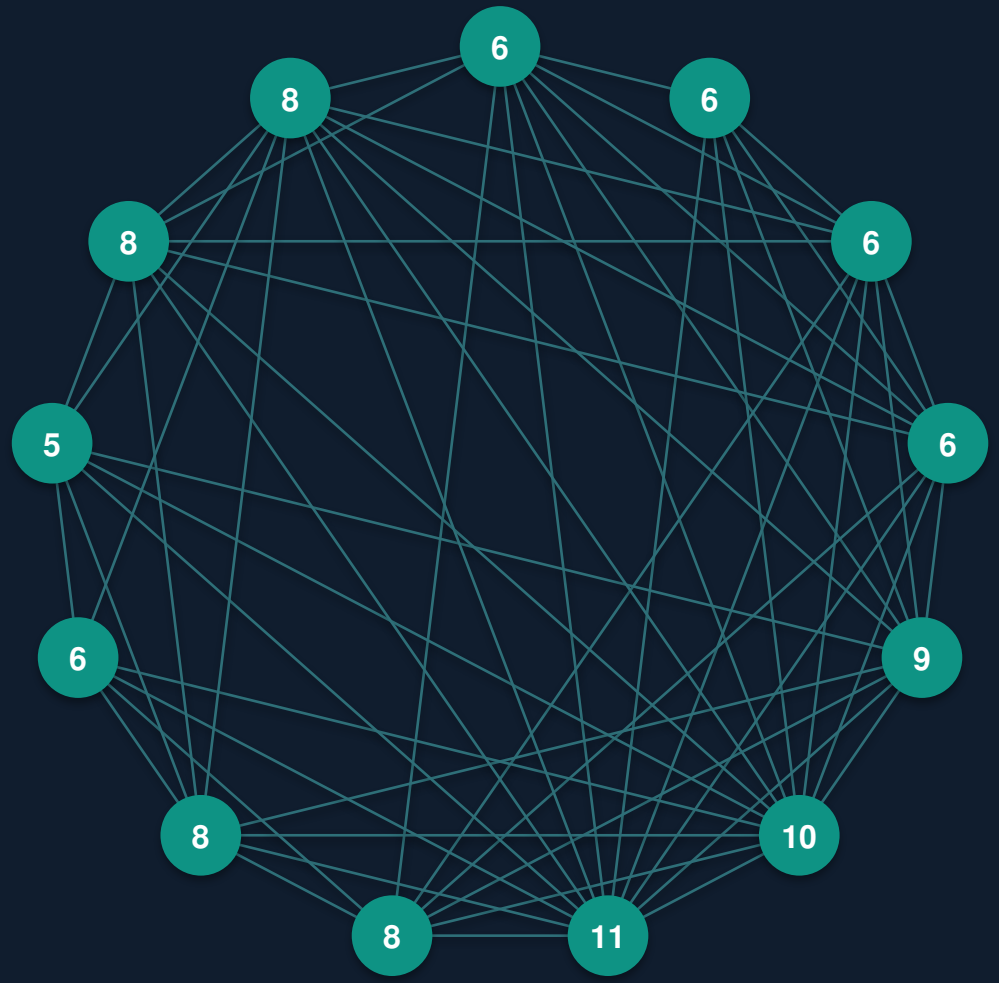
DHT · TLS 1.3 · туннели через ретрансляторы · сквозное шифрование · воспроизводимый эксперимент

Курсовая работа

Упрощённый уровень · Rust

N = 15 узлов · k = 3 · α = 3 · R = 3

2026



Граф таблиц маршрутизации, приёмка N = 13: число в узле — его контактов (из 12)

Что сделано и что подтверждено

30/30

поиск цели, которой нет в таблице инициатора

90/90

сообщений через 2 ретранслятора с подтверждением получателя

10/10

восстановлений после отказа ретранслятора, медиана 96 мс

0

вхождений открытого текста в захвате трафика (TLS 1.3 и AKE)

TCP + кадр 24 Б → **Kademlia DHT** → **Туннель + TLS 1.3 + E2E** → **Эксперимент N = 15 и 21**

Данные: experiments/results/*/20261102_* и */20261111_22* (N = 15 и 21), pcap_no_plaintext (TLS и AKE); цепочка — устные требования §7

Почему Kademlia, а не Chord или Pastry

	Kademlia	Chord	Pastry
Метрика	**XOR, симметричная**	кольцо, несимметричная	префиксы
Длина поиска	**O(log N)**	O(log N)	$O(\log_{2^b} N)$
Учится на запросах	**да**	нет — stabilize	частично
Параллельный поиск	**α = 3 сразу**	последовательно	последовательно
Отказ узла в пути	**обходит без ожидания**	тайм-аут	тайм-аут

Ключевое

Симметричная метрика: таблица наполняется из входящих запросов — стабилизация не нужна.

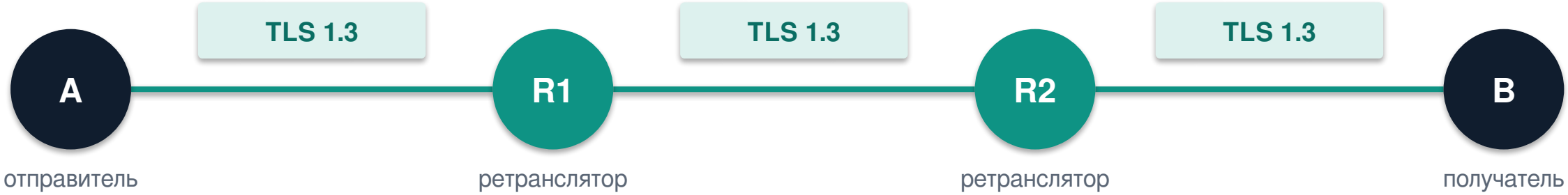
α параллельных запросов: мёртвый кандидат не задерживает поиск.

Проверено в эксплуатации: BitTorrent Mainline, IPFS/libp2p.

Maymounkov & Mazières 2002; Stoica et al. 2001; Rowstron & Druschel 2001 — записка, раздел 2

Два уровня защиты: канал и содержимое

E2E-конверт A → B: X25519 (эфемерный) + HKDF-SHA256 + AES-256-GCM, подпись Ed25519 внутри



Канал (hop-by-hop)

TLS 1.3, взаимная аутентификация, без 0-RTT и TLS 1.2. NodeID = SHA-256(ключ) — из сертификата пира.

Содержимое (end-to-end)

Ретрансляторы видят только конверт. Подпись связывает сообщение с NodeID отправителя.

Проверено

44 негативных теста: подмена ключа и подписи, изменённый шифртекст, replay, откат версии, TLS 1.2.

RFC 8446, RFC 7748, RFC 5869, NIST SP 800-38D; tests/test_tls.rs, test_e2e.rs, test_app_ack.rs; ADR-11, ADR-15

DHT: распределённый поиск и подписанные записи

1. **Bootstrap**
   PING известного адреса → self-lookup → публикация своей адресной записи

2. **Lookup**
   α = 3 параллельных FIND_NODE, ответы по мере прихода; стоп, когда ближе не стало

3. **STORE / FIND_VALUE**
   R = 3 ближайших узла, кворум 2; чтение — до первой записи с валидной подписью

4. **Поддержка**
   refresh — 60 с · republish — 80 с · TTL — 180 с

Подписанная запись

```
sig = Ed25519(владелец,
  ключ ‖ NodeID ‖ sequence ‖ значение)
```

NodeID = SHA-256(0x01 ‖ ключ) — без PKI

Хранитель отвергает

неверную подпись или пару NodeID–ключ
меньший sequence (откат версии)
чужой живой ключ (first-writer-wins)
адресную запись не под своим ключом

PROTOCOL.md §4; записка, разделы 7–8; tests/test_dht_records.rs, test_node_record.rs

Туннель: подтверждение и восстановление

1

TUNNEL_BUILD

к каждому relay; проверки: NodeID, max_hops,

next_hop ≠ инициатор, уникальный tunnel_id

2

TUNNEL_DATA

hops_left уменьшается на каждом relay;

при 0 кадр отбрасывается (петли)

3

Сквозной ACK

relay отвечает после следующего hop:

ACK первого relay = кадр у получателя

4

Отказ → новый маршрут

нет ACK → DEAD → lookup → туннель без отказавшего узла → повтор в той же отправке



PROTOCOL.md §5; ADR-17, ADR-21; tests/test_loops.rs, test_tunnel.rs

Стенд и план эксперимента

Underlay
1 машина, loopback — полная связность

Bootstrap
star / ring: узлу известен 1 адрес

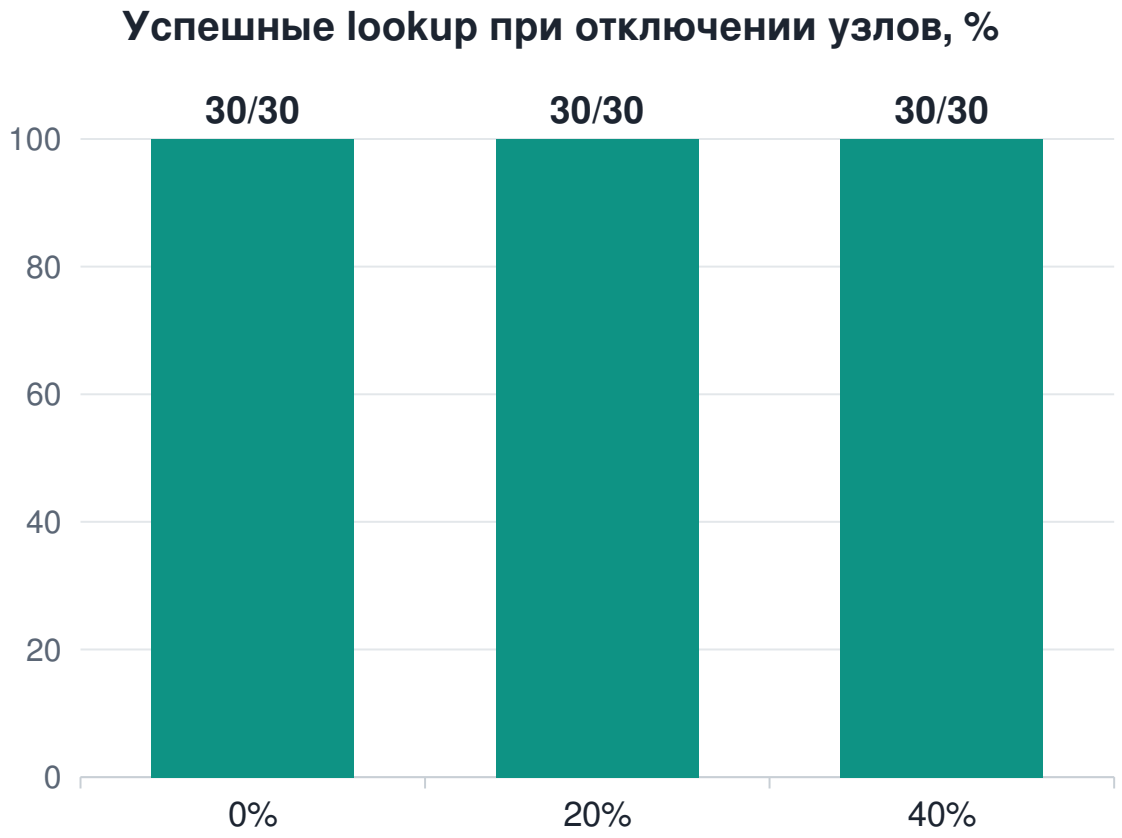
DHT
5–11 контактов при N−1 = 14

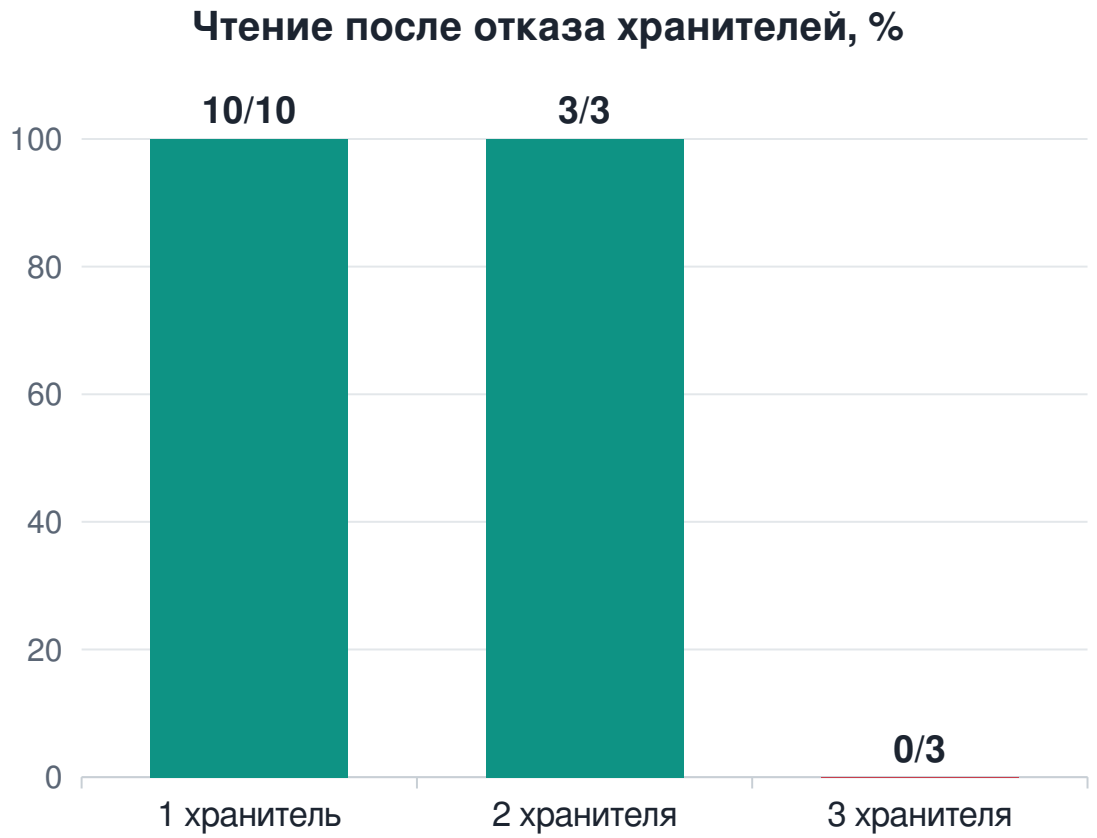
Туннельный
A → 2 relay → B, свой для каждой пары

Сценарий	Фактор	n
01 холодный старт	—	5 запусков
02 поиск	цель вне таблицы	30
03 STORE/FIND	—	30
04 отказ хранителя	1 / 2 / 3	10 + 3 + 3
05 отказ кандидатов	0 / 20 / 40%	3 × 30
06 сообщения	первое / сессия	30 / 60
07 отказ relay	1-й / 2-й	10
09 bootstrap	star / ring	5 + 5

N = 15, k = 3, α = 3, R = 3, seed = 1; запуск: bash experiments/run_all.sh; meta.json в каждом прогоне

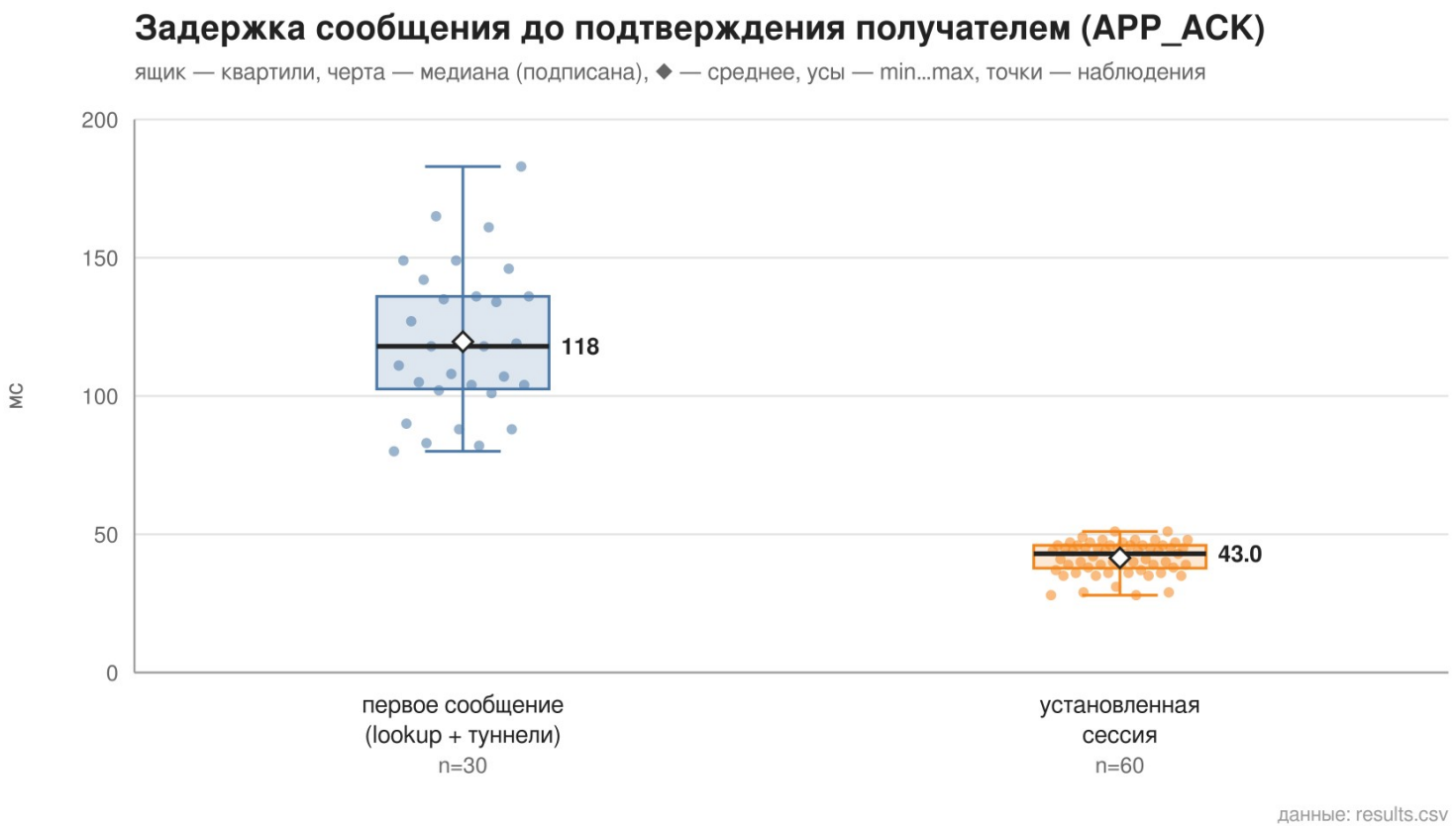
Поиск и хранение выдерживают отказы





05_lookup_failure/20261102_095108, 04_keeper_failure/20261102_095005 (results.csv)

Доставка: первое сообщение и установленная сессия



118 мс — первое сообщение, медиана (lookup + 2 туннеля)

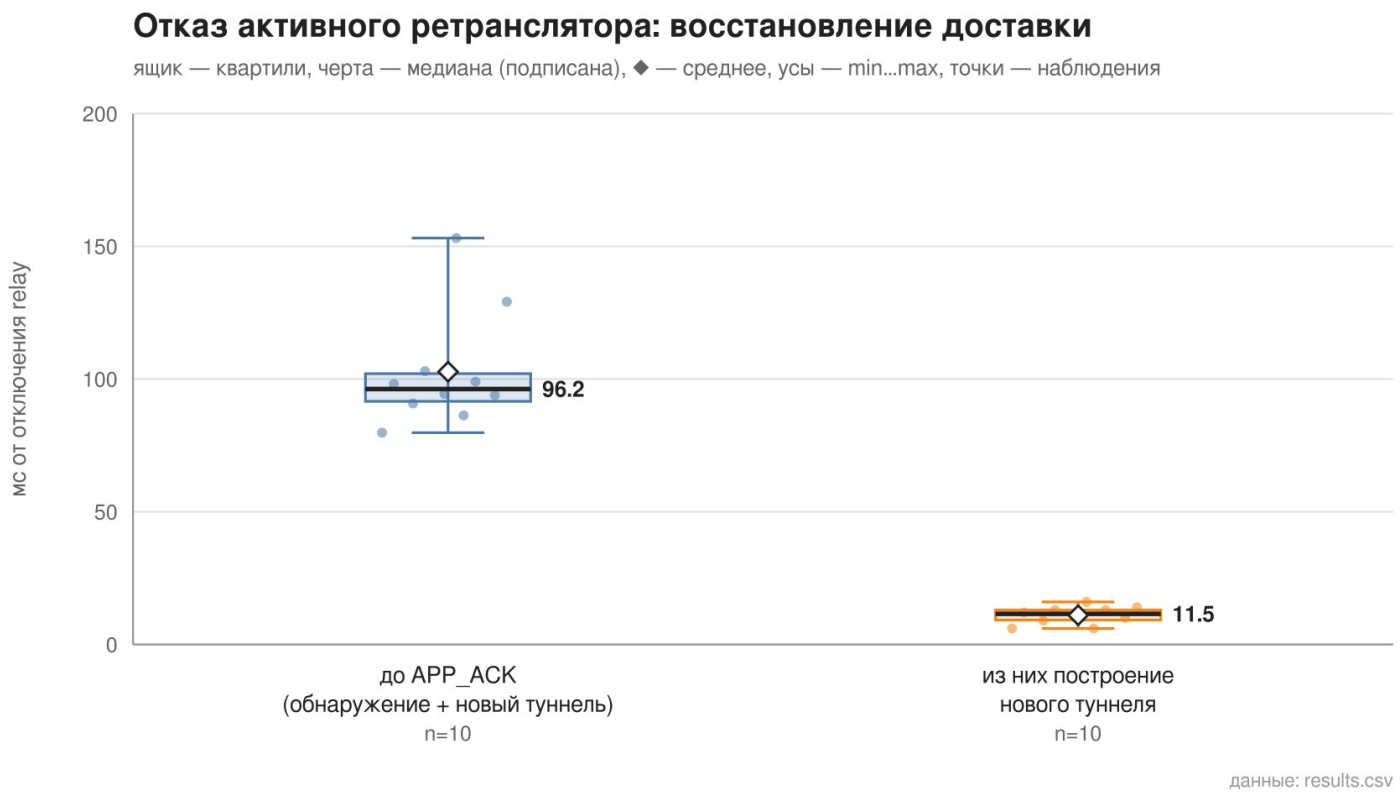
43 мс — установленная сессия, медиана, n = 60

10 мс — построение туннеля через 2 relay

3 мс — проход кадра по цепочке relay (соединения из пула)

06_tunnel_message/20261111_221343: 90/90 доставлено с APP_ACK; график — plots/rtt.svg из results.csv; debug-сборка

Отказ ретранслятора и автовосстановление



10/10 восстановлений, все в той же отправке

10/10 новый маршрут без отключённого узла

96 мс медиана от отключения до APP_ACK

11,5 мс из них — построение нового туннеля

07_relay_failure/20261111_221355: 10 инъекций (по 5 на 1-й и 2-й relay), SIGKILL; ADR-21

Продвинутый уровень; что не заявляется

Продвинутый уровень (N = 21)

1. **Пул туннелей**
   отказ relay → резервный туннель 10/10: восстановление 68 мс против 95 мс без пула

2. **Выбор по профилю кандидата**
   5 из 21 узла неисправны: 97% доставок без отказа против 67% (ближайшие) и 78% (случайные)

3. **Своё рукопожатие (AKE) и файлы**
   AKE = TLS в release (25 / 26 мс); файлы до 10 МиБ: SHA-256 30/30, служебная доля 1,4%

Ограничения

Одна машина: абсолютные задержки не переносятся на реальную сеть

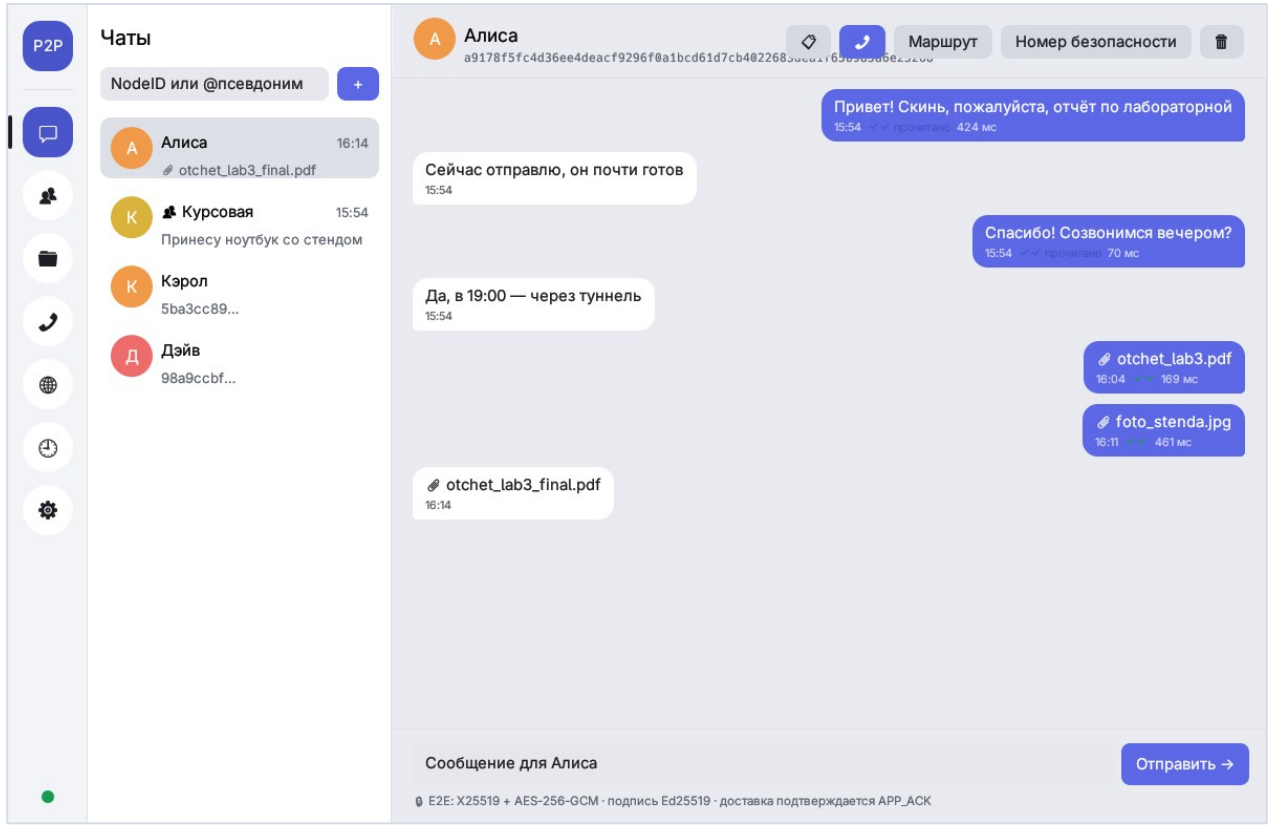
Netem и underlay: код есть, прогон требует root — не выполнялся

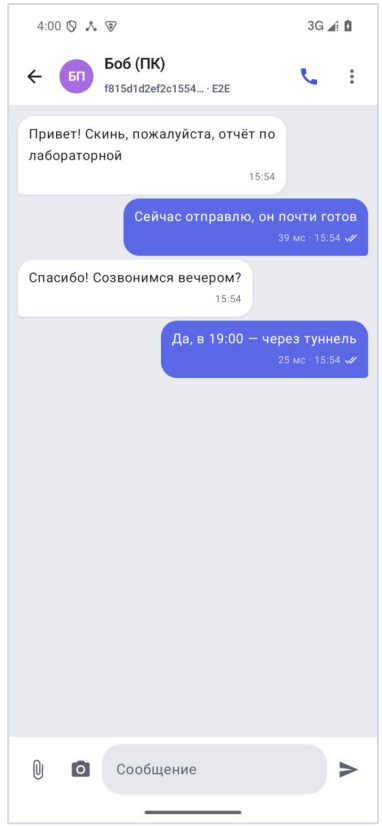
Не заявляется: анонимность, стойкость к Sybil/Eclipse, масштаб за N = 21

Нет: прямой секретности у получателя; кандидаты в relay — только из своей таблицы

*/20261111_22*_advanced: failover, relay_strategy, files_transfer, channel_compare (+ _release); дефекты, найденные экспериментом, — записка, раздел 13

Приложения: компьютер и телефон — те же узлы





Один p2p-node

ПК — процесс рядом с окном (egui); телефон — foreground-сервис, тот же узел под ARM

Интерфейс ↔ узел: HTTP API только на 127.0.0.1, с токеном

Телефон подключается по QR с компьютера

Чаты, группы, файлы, звонки, номер безопасности, офлайн-доставка через DHT

Скриншоты — стенд «Алиса / Боб» (docs/design); android/README.md, USERGUIDE §3–4; проверено: ПК + Android 15 (эмулятор)

Голосовые звонки через туннель

Микрофон
Opus 16 кГц, кадр 20 мс, 24 кбит/с, FEC

→

Ключ звонка
одноразовые X25519 в подписанных INVITE / ACCEPT → AES-256-GCM

→

Туннель
TUNNEL_MEDIA через ≥ 2 ретранслятора, без ACK, окно повторов

→

Приём
адаптивный джиттер-буфер, маскировка потерь

→

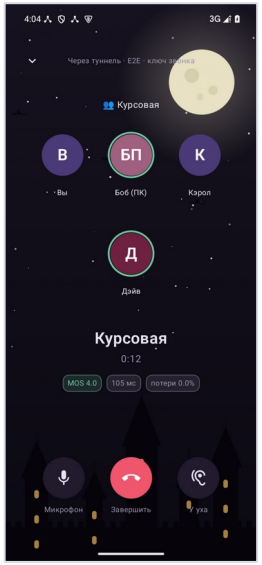
Группа до 5
каждый с каждым, смешивание в узле

62 мс установление (вызов → все участники), медиана, n = 15

68 мс «рот — ухо»: сеть + буфер + кодек; p95 80 мс (G.114: < 150)

0 % слышимых потерь; каждый слышит ровно собеседников — 15 / 15

4,1 MOS по E-модели (Ie = 10, Bpl = 20 — допущение для Opus)



voice_call/20261112_124521_release: 10 звонков 1:1 + 5 групповых, N = 15; call_stream: медиа p95 4 мс; ADR-39, 43, 44

Файлы: согласие, очередь, квоты

1

Предложение, а не поток
сначала имя, размер, SHA-256; без согласия на диск не пишется ни байта

2

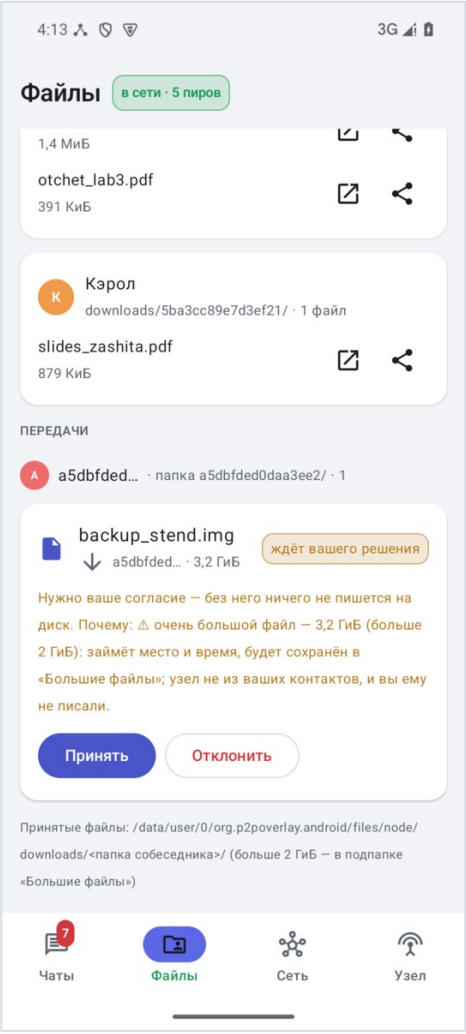
Кто решает
знакомый узел и ≤ 16 МиБ — сам; незнакомый или больше — вопрос; known / ask / auto в настройках

3

Пределы и очередь
2 приёма от одного узла, 8 всего — лишние ждут у отправителя; квота автоприёма 2 ГиБ, на 90 % — предупреждение

4

Хранение и имена
папка собеседника, > 2 ГиБ — «Большие файлы», права 0600; метка настоящего расширения, скрытые символы (U+202E) в имени исполняемого — предупреждение



ADR-45 … 48; tests/test_file_consent.rs, test_files.rs; files_transfer (advanced): служебная доля трафика 1,5 %

Защита от исчерпания ресурсов

Ресурс	Предел	От чего	Цена для пользователя
Входящие соединения	**1024 всего, 64 с одного IP**	одна машина занимает все	общий NAT: 30–60 узлов за адресом
Сообщения, вызовы	**5 в секунду, всплеск 30 — на NodeID**	спам	незаметно
Хранилище DHT	**доля 1/16 объёма; вытеснение, а не отказ**	один публикатор забивает соседа	нет
Маршруты relay	**64 на инициатора**	переполнение таблицы	нет
Файлы	**согласие, 2 / 8 приёмов, квота**	заливка диска	вопрос для незнакомых

Чего пределы не дают

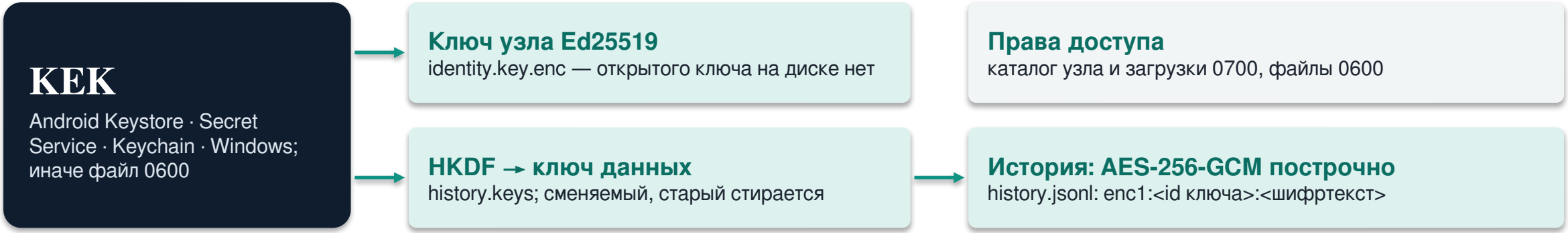
NodeID бесплатен: пределы «на узел» обходятся множеством ключей (Sybil).

Это дешёвая ограда от одной машины, а не защита от ботнета.

Можно добавить: головоломку или cookie под нагрузкой; после рукопожатия — предел по NodeID; в LAN — без предела.

THREAT_MODEL T27 – T38; ADR-45, 48, 49; docs/dev/SECURITY_IMPROVEMENTS.md — влияние на удобство и варианты

Данные на устройстве



Срок хранения — год

Удалить старое · перешифровать и оставить · **вынести в открытый файл**

Красная кнопка честная: экспорт нужен (архив, переезд), но опасен

Смена ключа данных

В любой момент: новый ключ, перешифровка, старый стирается.

Ключ узла и NodeID не меняются — собеседники ничего не замечают

Ошибки без адресов

В release ретранслятор называет следующий узел node:<16 hex> — без IP и причины.

Подробности — в журнале и --peer-errors detailed

ADR-47; tests/test_history_at_rest.rs, test_peer_errors.rs; desktop/src/secrets.rs, Android Keystore

Личность и доверие

Личность — NodeID

SHA-256 ключа Ed25519. Подделать нельзя без закрытого ключа

Адрес — только где узел сейчас (подписанная NodeRecord); каждое соединение доказывает владение ключом

Псевдоним — запись в DHT

Имя → NodeID, с подписью. Владеет первый записавший, пока запись жива (TTL 180 с)

Владелец выключен > 3 мин — имя может занять другой; добавленные контакты хранятся по NodeID

Номер безопасности

Сверка лично — как keysigning party (GnuPG) или «код безопасности» в Signal

До сверки — доверие при первой встрече (TOFU)

Почему ключ узла не меняется по расписанию

Подбор закрытого ключа по открытому — ≈ 2^{128} операций; возраст ключа атакующему не помогает. Раз в год меняется только ключ данных истории — в сети смены нет, нет и «окна» для подмены. Новый ключ узла = новый NodeID = другой узел. Чего нет: отзыва ключа при краже.

ADR-32, 47; SECURITY_IMPROVEMENTS: «Как узлы отличают друг друга», «Ежегодная смена ключа»

Проверка и воспроизводимость

197

автотестов
cargo test

65

негативных: подмена,
повтор, откат, квоты

17

сценариев: 9 из ТЗ
+ звук, файлы, пул, pcap

CI

узел, кросс-сборка,
APK, тесты в эмуляторе

Каждый прогон сохраняет

results.csv — сырые наблюдения; summary.md — медиана, p95

meta.json — коммит, конфигурация, seed, N / k / α / R

журналы узлов; графики строятся из CSV, а не руками

Найдено экспериментом и исправлено

поздние кадры звука на телефоне 5,8 % → адаптивный джиттер-буфер

гонка при входе в групповой звонок; потеря событий без подписчика

переполненная квота файлов молча превращалась в отказ → вопрос

cargo test (08_negative_crypto/20261112_194140_release); experiments/README.md; .github/workflows/android.yml

Дальше: прямая секретность и постквантовость

1. **Сменяемый подписанный ключ шифрования**
   запись в DHT раз в неделю, старый закрытый ключ стирается — кража ключа узла не раскроет старое

2. **Гибрид X25519 + ML-KEM-768**
   в той же записи — против «записать сейчас, расшифровать потом»

3. **Double Ratchet**
   новый ключ на каждое сообщение, как в Signal

	X25519	ML-KEM-768
Открытый ключ	32 Б	1184 Б
Добавка к конверту	32 Б	1088 Б
Скорость	десятки мкс	десятки мкс
Из ключа узла	выводится	нет — публиковать

Минусы ML-KEM у нас

короткое сообщение в ×6 больше на каждом переходе · подписи остаются Ed25519 · rustls + ring без ML-KEM (aws-lc-rs — сборка C под Android) · реализации молодые (KyberSlash) → только гибрид

FIPS 203 (ML-KEM, 2024); Signal X3DH / PQXDH, Double Ratchet; SECURITY_IMPROVEMENTS п. 2 и «ML-KEM: минусы»

Выводы

- Поиск без центрального каталога: 30/30, 3 итерации; 100% при отключении 40% узлов
- Записи на 4 узлах (автор + 3 реплики) переживают отказ 2 хранителей из 3
- TLS 1.3 или своё рукопожатие, E2E-конверт: открытого текста в трафике нет; 65 негативных тестов
- Доставка только через relay с подтверждением: первое сообщение 118 мс, сессия 43 мс
- Отказ relay → восстановление 10/10 за 96 мс; с пулом туннелей — 68 мс
- Приложения для ПК и Android — те же узлы; звонок: «рот — ухо» 68 мс, MOS 4,1
- Пределы ресурсов, согласие на файлы, шифрование на диске; границы названы

Воспроизведение: `bash experiments/run_all.sh` · данные, графики и журналы — experiments/results/